



Elliptic Curves over Finite Fields and the Discrete Logarithm Problem:

A Cryptographic Voting Protocol and its Post-Quantum Analysis

Final Evaluation | MA-400

Angel Sharma

Enrol. No. 22323005

Under Supervision of
Prof. Arunava Mandal



Table of Contents



1. Mid-Eval Recap
2. Why Post-Quantum
3. Lattice-Based Cryptography
4. Privacy-Preserving Tally
5. Results
6. Impact
7. Conclusion
8. References



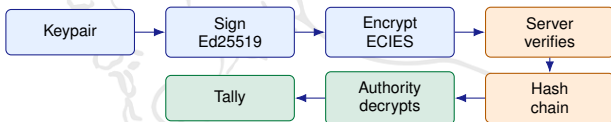
Mid-Eval Recap: What Was Built



A working, end-to-end departmental voting system on Curve25519.

Stack: Python + Flask backend (`libsodium`); React frontend (Register / Vote / Admin / Results); append-only ballot box with SHA-256 hash chain.

What a voter actually does



Outcome: every ballot is signed, encrypted, and hash-chained; the authority opens the box only after polling closes.

How It Works — The Same Crypto That Runs Today's Apps



All three primitives live on Curve25519 ($p = 2^{255} - 19$).

Primitive	Role in a ballot	Deployed in
Ed25519	authenticates the voter	SSH, Signal, iMessage
X25519 / ECIES	hides the vote	TLS 1.3, WhatsApp, WireGuard
SHA-256 chain	makes tampering visible	Bitcoin, Git, X.509

Impact. The prototype inherits 25 years of audited libraries and standardisation — the same stack billions of devices use daily.

Performance, Security & Why We Looked Beyond ECC



Measured on this laptop (50 runs):

Ed25519 sign	~0.10 ms	Public key	32 B
ECIES encrypt	0.14 ms	Ciphertext	71 B
ECIES decrypt	0.13 ms	30-voter election	< 8 ms

Security: ~ 128 -bit. Breaking one key needs $\sim 10^{38}$ ops — every computer on Earth for the age of the universe is not enough.

So why a post-quantum redesign? **Shor (1994):** a large quantum computer solves the discrete log in polynomial time — the 10^{38} wall collapses to minutes (“harvest now, decrypt later”).

Two Gaps Identified $\Rightarrow$ Post-Mid-Eval Work



Gap 1 — A quantum computer can break it

A future quantum computer running Shor's algorithm (1994) would crack the math behind ECC. *All three pieces — signatures, key exchange, and encryption — break together.*

Gap 2 — The authority sees every single vote

ECIES cannot add encrypted votes together. To get the count, the authority must open *every ballot one by one* — meaning it learns who voted for whom. The whole system rests on trusting one person.

**Lattice-based cryptography fixes both at once:
it survives quantum computers, *and* ballots can be added without
being opened.**



Lattice. For linearly independent $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^n$,

$$\mathcal{L} = \left\{ \sum_{i=1}^n z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\}.$$

The Shortest Vector Problem (SVP) is conjectured hard, **even for quantum computers**.

Learning With Errors (Regev, 2005). Fix prime q , dimension n , small error distribution χ . Given m samples

$$(\mathbf{a}_i, b_i), \quad \mathbf{a}_i \sim \mathbb{Z}_q^n, \quad e_i \sim \chi, \quad b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \pmod{q},$$

recover the secret $\mathbf{s} \in \mathbb{Z}_q^n$.

Why LWE is special

Regev's worst-case-to-average-case reduction: solving *random* LWE instances yields a *quantum* algorithm for SVP on *every* n -dim lattice.

Regev's Encryption Scheme — The Recipe



Step 1 — Key generation. Pick a secret vector $\mathbf{s}$ and a public matrix A . Add tiny noise $\mathbf{e}$:

$$\mathbf{b} = A\mathbf{s} + \mathbf{e} \pmod{q}, \quad \text{pk} = (A, \mathbf{b}), \quad \text{sk} = \mathbf{s}.$$

Without the noise, anyone could solve for $\mathbf{s}$ — the noise is what makes it secure.

Step 2 — Encrypt μ . Pick a random 0/1 vector $\mathbf{r}$; send

$$c_1 = A^T \mathbf{r}, \quad c_2 = \langle \mathbf{b}, \mathbf{r} \rangle + \mu \Delta, \quad \Delta \approx q/2.$$

Step 3 — Decrypt. Compute

$$v = c_2 - \langle \mathbf{s}, c_1 \rangle = \underbrace{\text{small noise}}_{< \Delta/2} + \mu \Delta.$$

The secret $\mathbf{s}$ cancels the randomness; rounding v/Δ recovers μ .



Ciphertexts add componentwise:

$$(c_1, c_2) + (c'_1, c'_2) = (c_1 + c'_1, c_2 + c'_2) \pmod{q}.$$

Decryption of the sum gives

$$(\mu + \mu')\Delta + (\langle \mathbf{e}, \mathbf{r} \rangle + \langle \mathbf{e}, \mathbf{r}' \rangle).$$

What happens when we add many ballots? Each ciphertext carries a tiny bit of noise. When we add N ballots, the noise piles up — but slowly, roughly like $\sqrt{N}$. So we have room to spare.

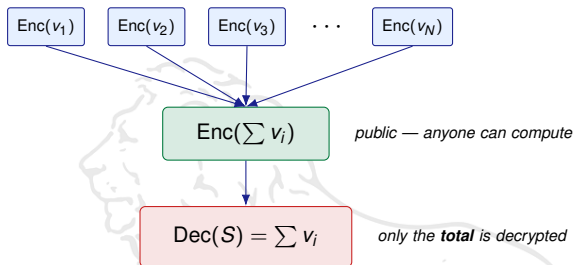
*With the parameters we picked, we can safely sum about **200 ballots per candidate** before the noise interferes with the answer.*

The cryptographic shift

ECIES: ballot $\rightarrow$ *decrypt every one* $\rightarrow$ tally.

Regev: **sum ciphertexts** $\rightarrow$ *decrypt only the total*. Individual votes are never seen.

Privacy-Preserving Tally — The Substantive Upgrade



What the authority learns under each scheme

	ECIES (mid-eval)	Regev (final)
Per-ballot plaintext seen by authority?	yes, every one	no, never
Tally requires per-ballot decryption?	yes	no
Trust model	"trusted authority"	"cannot link votes"

Benchmarks — ECC vs. Lattice (50 iters)



Operation	ECC (X25519/ECIES)	LWE (Regev)
Key generation — mean (ms)	0.119	183.1
Encrypt — mean (ms)	0.137	1.943
Decrypt — mean (ms)	0.132	0.007
Public key (bytes)	32	394,752
Secret key (bytes)	32	768
Ciphertext (bytes)	71	771

- ❑ ECC is constant-time C (`libsodium`); Regev is pure-Python for the dissertation. The $1000\times$ keygen gap is a *language* gap.
- ❑ The $12,336\times$ public-key gap is **intrinsic** to plain Regev.
- ❑ NIST's **ML-KEM (Kyber)** uses Module-LWE to shrink the public key to ~ 1 KB while keeping the same hardness assumption.

End-to-End Simulated Election (30 Voters, 3 Candidates)



Stage	ECC	Regev LWE
Authority key setup (ms)	0.07	182.7
Cast all 30 ballots (ms)	4.02	185.3
Tally + decrypt (ms)	3.90	0.73
Individual ballots decrypted?	yes (every one)	no (totals only)
Result matches ground truth?	✓	✓

Tally output (both schemes agree with truth):

Candidate	True	ECC	Regev
Prof. Alice	9	9	9
Prof. Bob	8	8	8
Prof. Carol	13	13	13

Tally is $5\times$ faster on Regev — and reveals nothing about individual votes.

Does It Scale? Projection to Real Electorates



India general election — 970 million voters:

Scheme	Ballot box	PK dist.	Tally (1 core)	Tally (1000 cores)
ECC (X25519 / ECIES)	192 GB	29 GB	4.4 days	6.4 min
Regev LWE (this dissertation)	2.0 TB	348 TB	6.5 h	23.6 s
Kyber-768 (NIST ML-KEM)	2.9 TB	1.0 TB	6.5 h	23.6 s

Three observations

- ❑ **The voter's phone barely notices.** Casting a ballot takes about 2 ms on every scheme.
- ❑ **The only number that looks scary is the public key.** Plain Regev needs ~ 400 KB per voter. The NIST standard **Kyber** shrinks it to about 1 KB
- ❑ **Surprise: counting votes is faster on lattices.** actually saves time and a lot of money



Mathematical

- ❑ Self-contained development of $E(\mathbb{F}_p)$ as a finite abelian group; ECDLP hardness; Regev's reduction sketched explicitly.
- ❑ Two hardness assumptions exercised end-to-end on the *same* application: ECDLP and LWE.

Engineering

- ❑ Production-style ECC voting backend (Flask + React) on Curve25519.
- ❑ From-scratch Regev LWE library in pure Python (keygen / enc / dec / homomorphic add / tally)

Cryptographic

- ❑ Hybrid security: secure if *at least one* of ECDLP or LWE remains hard.
- ❑ Voter privacy upgraded from “trust the authority” to “**authority cannot link ballots to voters even in principle**”.



- ❑ Elliptic-curve cryptography gives an elegant, deployable voting protocol — but offers *zero* post-quantum security and forces the authority to see every vote.
- ❑ Lattice-based encryption (Regev LWE) closes both gaps with one tool: quantum-hard *and* additively homomorphic.
- ❑ A working prototype shows the lattice rewrite is feasible today; with Module-LWE (Kyber / ML-KEM) it extends to planetary scale.
- ❑ **The outcome is a voting system whose confidentiality is expected to hold for the next 30 years at least** — well beyond credible timelines for a cryptanalytically relevant quantum computer.



- [1] J. Hoffstein, J. Pipher, J. H. Silverman. *An Introduction to Mathematical Cryptography*, 2nd ed., Springer, 2014.
- [2] P. W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer." *SIAM J. Comput.* 26(5):1484–1509, 1997.
- [3] O. Regev. "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography." *J. ACM* 56(6), 2009.
- [4] NIST. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203, August 2024.
- [5] NIST. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, August 2024.
- [6] R. Avanzi *et al.* *CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation*, v3.02. 2021.
- [7] A. Langley, M. Hamburg, S. Turner. *Elliptic Curves for Security*. IETF RFC 7748, 2016.
- [8] V. Lyubashevsky, C. Peikert, O. Regev. "On Ideal Lattices and Learning with Errors over Rings." *EUROCRYPT 2010*.
- [9] S. Josefsson, I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. IETF RFC 8032, 2017.
- [10] NIST. *Secure Hash Standard (SHS)*. FIPS 180-4, August 2015.



- [11] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007.
- [12] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. IETF RFC 5869, 2010.
- [13] E. Barker. *Recommendation for Key Management, Part 1: General*. NIST SP 800-57 Pt. 1 Rev. 5, 2020.
- [14] J. M. Pollard. “Monte Carlo Methods for Index Computation (mod p).” *Math. Comp.* 32(143):918–924, 1978.
- [15] A. K. Lenstra, H. W. Lenstra Jr., L. Lovász. “Factoring Polynomials with Rational Coefficients.” *Math. Ann.* 261:515–534, 1982.
- [16] C.-P. Schnorr, M. Euchner. “Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems.” *Math. Programming* 66:181–199, 1994.
- [17] M. Ajtai. “The Shortest Vector Problem in L_2 is NP-hard for Randomized Reductions.” *STOC 1998*, pp. 10–19.
- [18] L. K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search.” *STOC 1996*, pp. 212–219.
- [19] G. Pope. *kyber-py: A Pure-Python Reference Implementation of CRYSTALS-Kyber and ML-KEM (FIPS 203)*. GitHub, accessed 2026.

Thank You.

Open for Questions.